Online Adaptive Differentiable Predictive Control With Application to IPMSM MTPA Control

1st Changeun Park
CCS Graduate School of Mobility
KAIST
Daejeon, Republic of Korea
aegis3141@kaist.ac.kr

2nd Seunghun Jang
CCS Graduate School of Mobility
KAIST
Daejeon, Republic of Korea
shjang7071@kaist.ac.kr

3rd Kyunghwan Choi
CCS Graduate School of Mobility
KAIST
Daejeon, Republic of Korea
kh.choi@kaist.ac.kr

***Abstract*—Nonlinear model predictive control (NMPC) handles nonlinear dynamics and constraints through repeated optimization, but its computational burden can limit high-frequency implementation. To alleviate the computational burden of real-time control while retaining online adaptability, this paper proposes an online adaptive differentiable predictive control (DPC) framework that updates a residual neural policy without offline pretraining or NMPC-generated targets. A baseline feedforward controller provides nominal control actions, allowing the residual neural network to focus on corrective actions. The residual policy is optimized directly using predictive-cost gradients from short rollouts initialized by recent operating samples. A differentiable HardNet-Aff layer is incorporated to enforce affine control constraints without relying on soft constraint-violation penalties. Control execution and policy learning are performed in separate processes to exclude backpropagation from the time-critical control path. The proposed framework is evaluated through interior permanent magnet synchronous motor (IPMSM) current-control simulations under repeated maximum torque per ampere (MTPA) references. Online adaptation, current-tracking performance, and computational efficiency are compared with acados-based NMPC under varying reference and model conditions.**

***Index Terms*—nonlinear model predictive control, differentiable predictive control, online learning, residual policy learning, constraint handling, IPMSM**

I. Introduction

Modern control systems are increasingly required to maintain high control performance under diverse operating conditions and constraints. In such systems, nonlinear model predictive control (NMPC) is one of the promising control approaches. By predicting future system behavior and repeatedly solving constrained optimization problems at each control step, NMPC provides a systematic framework for satisfying complex control requirements. However, as NMPC applications expand to systems with limited computational resources and short sampling periods, this repeated optimization has become a computational bottleneck. In particular, the need to solve nonlinear optimization problems at every control step can limit the real-time implementation of NMPC on resource-constrained platforms, such as microcontroller units (MCUs).

To address this challenge, various approaches have been developed to reduce the computational burden of NMPC. Real-time iteration (RTI) performs a single sequential quadratic programming (SQP) iteration per control step [1], while explicit MPC precomputes the optimal control law [2]. Neural network (NN)-based methods have also been employed for direct policy approximation [3] and solver-assisted acceleration [4]. However, RTI and solver-assisted approaches still depend on online optimization, whereas explicit MPC and NN-based policy approximation can require substantial offline computation or training.

Meanwhile, differentiable predictive control (DPC) enables direct NN policy optimization through predictive-cost gradients without expert-generated optimal control inputs [5]. However, conventional DPC approaches often rely on offline training and soft constraint penalties. Although online adaptation is theoretically possible, gradient-based policy updates can introduce computational overhead, while an initially untrained policy may exhibit poor control performance and constraint violations. Moreover, soft penalties alone do not guarantee constraint satisfaction. These limitations motivate an online learning framework that maintains a nominal control action and explicitly handles constraints during adaptation.

To address these challenges, this study proposes an online adaptive DPC framework combining a baseline controller, a residual NN, and HardNet-Aff [6]. The baseline controller provides a nominal control input, allowing the residual NN to focus on learning corrective actions. HardNet-Aff incorporates affine constraints derived from a continuous-time control barrier function (CBF), reducing reliance on soft constraint penalties. The residual policy is updated directly through predictive-cost gradients from rollouts initialized by recent operating samples, without offline pretraining or NMPC-generated targets. Control execution and policy learning are performed in separate processes to exclude backpropagation from the time-critical control path.

Since online policy adaptation requires repeated updates, recurring operating conditions provide opportunities for gradual policy improvement. To investigate this capability, the proposed framework is applied to IPMSM current control under repeated maximum torque per ampere (MTPA) current references. Its online adaptation, current-tracking performance, and computational efficiency are evaluated against acados-based NMPC [8] under varying reference conditions and model mismatch.

II. Preliminaries

A. Differentiable Predictive Control

Differentiable predictive control (DPC) optimizes a neural control policy by differentiating a predictive control objective through the closed-loop system dynamics [5]. For each initial condition, the predicted trajectory is generated by recursively applying the NN policy and a differentiable discrete-time system model:

$$\begin{aligned} u_k^{(i)} &= \pi_\theta(\xi_k^{(i)}), \\ x_{k+1}^{(i)} &= f_d(x_k^{(i)}, u_k^{(i)}), \quad k = 0, \dots, N-1, \end{aligned} \tag{1}$$

where $x_0^{(i)}$ is the initial state of the i-th rollout, $\xi_k^{(i)}$ denotes the policy features, and f_d is the discrete-time prediction model.

Based on [5], the DPC loss is formulated as

$$\begin{aligned} \mathcal{L}_{\mathrm{DPC}}(\theta) = \frac{1}{mN} \sum_{i=1}^{m} \Bigg[\sum_{k=0}^{N-1} \Big(q_\ell \ell_k^{(i)} + q_x p(c_{x,k}^{(i)}) \\ + q_u p(c_{u,k}^{(i)}) \Big) + q_N p_N(x_N^{(i)}) \Bigg], \end{aligned} \tag{2}$$

where m is the number of rollouts, N is the prediction horizon, $\ell_k^{(i)} = \ell(x_k^{(i)}, u_k^{(i)}, r_k^{(i)})$ is the stage cost, and p_N is the terminal penalty. The scalar coefficients q_ℓ, q_x, q_u, and q_N denote the corresponding weights.

The terms $c_{x,k}^{(i)} \le 0$ and $c_{u,k}^{(i)} \le 0$ represent state and input constraint residuals, respectively. Their violations are evaluated using a common penalty function:

$$p(c) = \frac{1}{n_c} \| \operatorname{ReLU}(c) \|_2, \tag{3}$$

where c is a constraint residual vector and n_c is its dimension.

The predicted states and inputs depend on the NN parameters through the closed-loop rollout. Gradients are propagated through the system model and NN policy, enabling direct policy optimization without expert-generated control targets.

B. HardNet-Aff

HardNet-Aff handles input-dependent affine constraints through a differentiable closed-form projection of the control output [6]. For a single affine constraint $a(x)^\top u \le b(x)$, the projected output of an unconstrained control action $\tilde{u}$ is given by

$$\begin{aligned} \mathcal{H}_{\mathrm{Aff}}(\tilde{u}; x) = \tilde{u} - \frac{a(x)}{\|a(x)\|_2^2} \\ \cdot \operatorname{ReLU}\left(a(x)^\top \tilde{u} - b(x) \right), \end{aligned} \tag{4}$$

where $a(x) \neq 0$. This projection preserves feasible control actions and maps infeasible actions onto the constraint boundary. The mapping is differentiable almost everywhere, allowing gradient-based policy optimization.

A continuous-time control barrier function (CBF) provides one way to construct such an affine constraint [9]. For the safe set defined by $h(x) = I_{\max}^2 - \|x\|_2^2$ and control-affine dynamics $\dot{x} = f(x) + B(x)u$, the CBF condition $\dot{h}(x,u) + \alpha h(x) \ge 0$, with $\alpha > 0$, becomes

$$\begin{aligned} g(x)^\top u + c(x) \ge 0, \\ g(x) = -2B(x)^\top x, \\ c(x) = -2x^\top f(x) + \alpha h(x). \end{aligned} \tag{5}$$

Thus, the CBF constraint can be incorporated into HardNet-Aff by setting $a(x) = -g(x)$ and $b(x) = c(x)$.

III. Proposed Method

A. Online Adaptive Differentiable Predictive Control

The proposed controller combines a baseline controller, a residual NN, and HardNet-Aff. The control policy is defined as

$$\begin{aligned} u_{\mathrm{base},k} &= u_{\mathrm{ff}}(r_k), \\ \tilde{u}_k &= u_{\mathrm{base},k} + \pi_\theta(\xi_k), \\ u_k &= \mathcal{H}_{\mathrm{Aff}}(\tilde{u}_k; x_k), \end{aligned} \tag{6}$$

where x_k, r_k, and u_k denote the system state, reference, and control input, respectively. Here, $u_{\mathrm{base},k}$ is the nominal control action generated by the feedforward law u_{ff}, and ξ_k is the NN feature vector containing state and reference information. The combined action $\tilde{u}_k$ is processed by HardNet-Aff to obtain the applied input u_k.

For efficient online updates, a single-hidden-layer NN with fixed random hidden parameters and direct connections [7] is employed:

$$\begin{aligned} \pi_\theta(\xi_k) = W_{\mathrm{nl}} \tanh(W_h \xi_k + b_h) \\ + W_{\mathrm{lin}} \xi_k + b_o. \end{aligned} \tag{7}$$

Only $\theta = \{W_{\mathrm{nl}}, W_{\mathrm{lin}}, b_o\}$ is trained, while W_h and b_h remain fixed. The output parameters are initialized to zero, so the initial raw policy equals the baseline controller.

Unlike conventional penalty-based DPC, the proposed framework incorporates HardNet-Aff to enforce the specified affine input constraints without soft constraint-violation penalties. The baseline controller provides a nominal control action from initialization, allowing the lightweight residual NN to focus on corrective actions. By separating nominal control and constraint handling from residual learning, the proposed framework enables online adaptation without offline pretraining.

The residual NN is trained by minimizing a predictive objective consisting of tracking, input-variation, and terminal costs:

$$\begin{aligned} J_N(\theta; \zeta) = \frac{1}{2} \sum_{j=0}^{N-1} \Big(\|\hat{x}_j - r_j\|_Q^2 \\ + \|\hat{u}_j - p_j\|_{R_{\Delta u}}^2 \Big) + \frac{1}{2} \|\hat{x}_N - r_N\|_{Q_N}^2. \end{aligned} \tag{8}$$

where p_j denotes the previous control input, and ζ represents the rollout initial conditions and reference information. The

TABLE I
IPMSM SIMULATION PARAMETERS

Parameter	Value
Sampling time T_s	0.5 ms
Stator resistance R_s	0.326 Ω
Pole pairs p	3
Mechanical speed	500 rpm

predicted trajectories are generated using the complete policy in (6), including HardNet-Aff. Gradients of the predictive cost are propagated through the model and policy to update the residual NN without NMPC-generated control targets.

IV. SIMULATION RESULTS

A. Simulation Setup

The IPMSM plant is modeled using measured d–q flux maps of a 2.2-kW IPMSM, $\lambda(x) = [\lambda_d(x), \lambda_q(x)]^\top$, with the flux Jacobian $J_\lambda(x) = \partial\lambda/\partial x$. The current dynamics are

$$\dot{x} = J_\lambda(x)^{-1} \begin{bmatrix} v_d - R_s i_d + \omega_e \lambda_q(x) \\ v_q - R_s i_q - \omega_e \lambda_d(x) \end{bmatrix}, \tag{9}$$

where $\omega_e = p\omega_m$. The flux maps are interpolated using two-dimensional splines, and the plant is discretized using forward Euler. The nominal simulation parameters are summarized in Table I.

Three scenarios are considered to evaluate online adaptation under different operating conditions. The Easy scenario uses six current-magnitude references of $[4, 6.5, 9, 11.5, 7, 12.5]$ A. The Hard scenario uses $[4, 14.2, 16, 11.5, 14.8, 12.5]$ A, including a reference exceeding the 15-A current limit. The Model Mismatch scenario uses the Easy references with a 10% increase in plant resistance while retaining the nominal prediction model.

The corresponding d–q current references are generated using the MTPA method of [10] and filtered by a 50-Hz low-pass filter. Each reference level is held for 0.25 s, forming a 1.5-s cycle.

Both controllers use a 10-step prediction horizon with $Q = I_2$, $R_{\Delta u} = 10^{-4} I_2$, and $Q_N = 2I_2$. The proposed controller employs HardNet-Aff with $\alpha = 1500\,\mathrm{s}^{-1}$, whereas acados NMPC imposes current-circle constraints and uses SQP-RTI with HPIPM [8]. The residual NN contains 200 fixed hidden nodes and 414 trainable parameters. Online adaptation uses 10 rollout initial conditions per update, with Adam learning rates of 3×10^{-4} for Easy and Model Mismatch and 3×10^{-3} for Hard.

B. Online Policy Adaptation

Separate residual NN policies were trained for 20,000 asynchronous Adam updates in the Easy, Hard, and model-mismatch scenarios. The proposed method - initial (W_0)/ 1,000 update (W_{1000}) / 20,000 update (W_{20000}) policies were frozen and evaluated over one identical 1.5-s reference cycle. Table II reports the root-mean-square current tracking error, $\mathrm{RMSE} = \sqrt{\mathrm{mean}[(i_d - r_d)^2 + (i_q - r_q)^2]}$, relative to the 50-Hz-filtered references.

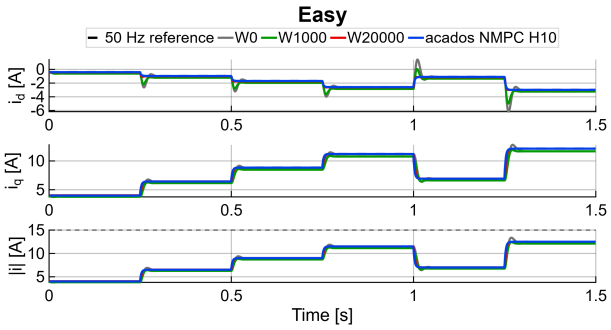


Fig. 1. Easy scenario: current tracking with the initial and adapted policies and the acados NMPC benchmark.

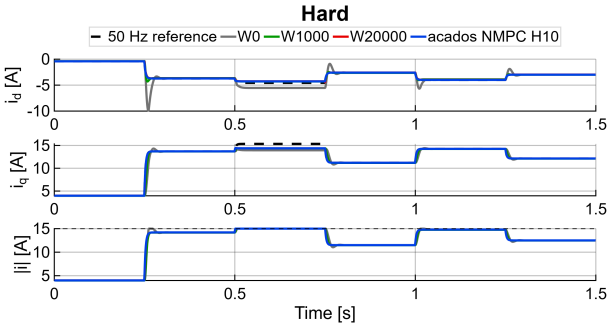


Fig. 2. Hard scenario: current tracking with a reference segment exceeding the nominal 15-A current limit.

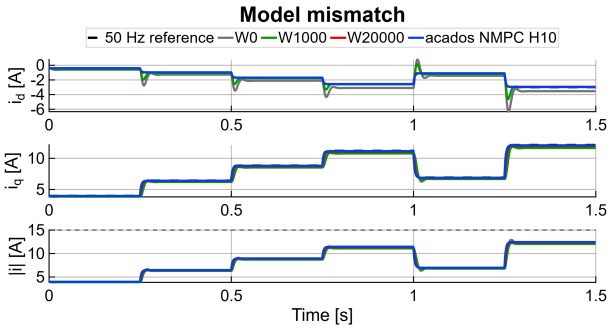


Fig. 3. Model-mismatch scenario: current tracking with 10% higher stator resistance in the plant than in the prediction model.

TABLE II
OVERALL CURRENT TRACKING RMSE (A)

	Proposed			
Scenario	W_0	W_{1000}	W_{20000}	NMPC
Easy	0.5328	0.5888	0.1226	0.00049
Hard	0.9947	0.4946	0.4066	0.4030
Model mismatch	0.7441	0.5041	0.1628	0.0361

After 20,000 updates, RMSE decreased by 77.0%, 59.1%, and 78.1% relative to proposed(W_0) in the Easy, Hard, and model-mismatch scenarios, respectively. Figs. 1– 3 show improved tracking in all three cases, with W_{20000} approaching the NMPC tracking performance relative to the initial policy.

C. Computational Performance

Control computation times were measured over 3,000 steps per scenario using the frozen W_{20000} policy. The measurement environment is summarized in Table III.

TABLE III
COMPUTATION TIME MEASUREMENT ENVIRONMENT

Item	Specification
Platform	HP OMEN 16-n0xxx
CPU	AMD Ryzen 7 6800H (8C/16T)
Memory	15.21 GiB available
OS	Windows 11
Software	MATLAB R2026a
Measurement	CPU wall-clock time (no GPU)

As shown in Table IV and Fig. 4, the proposed controller achieved average computation times of 0.146–0.174 ms, compared with 0.427–0.447 ms for acados NMPC, corresponding to a 2.45–3.06-fold speedup.

TABLE IV
CONTROL COMPUTATION TIME OVER 3,000 STEPS

Scenario	Controller	Mean (ms)	95th (ms)	> 0.5 ms
Easy	Proposed	0.174	0.306	3
	NMPC	0.427	0.645	375
Hard	Proposed	0.146	0.157	0
	NMPC	0.447	0.723	473
Mismatch	Proposed	0.149	0.178	0
	NMPC	0.443	0.687	479

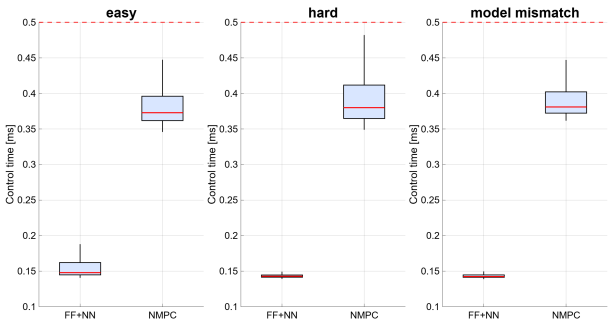


Fig. 4. Control computation time distributions. The vertical axis is limited to 0.5 ms; longer execution times are not displayed.

V. DISCUSSION AND CONCLUSION

This study demonstrated that online residual policy learning improves a flux-map feedforward controller without offline pretraining or NMPC-generated targets. After 20,000 updates, tracking RMSE decreased by 77.0%, 59.1%, and 78.1% relative to the initial policy in Easy, Hard, and model mismatch, respectively. These results suggest that predictive-cost gradients enable the residual NN to improve the baseline action. The learned tracking performance moved closer to that of NMPC in all three scenarios.

Compared with acados SQP-RTI, the proposed controller achieved a 3-fold reduction in mean control computation time. The policy avoids online numerical optimization, while asynchronous learning keeps backpropagation outside the control path. The nominal 15-A current limit was respected in most evaluated cases; however, Hard case exhibited a maximum exceedance of 1.325 mA (0.0088%) over 25 samples, potentially associated with applying a continuous-time CBF in a sampled system.

Future work will focus on accelerating online policy adaptation, incorporating discrete-time CBF constraints through HardNet-CVX, and extending the proposed framework to actor–critic reinforcement learning with multi-step lookahead to further investigate adaptability, computational efficiency, and constraint satisfaction.

REFERENCES

[1] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM J. Control Optim.*, vol. 43, no. 5, pp. 1714–1736, 2005, doi: 10.1137/S0363012902400713.

[2] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, "The explicit linear quadratic regulator for constrained systems," *Automatica*, vol. 38, no. 1, pp. 3–20, 2002, doi: 10.1016/S0005-1098(01)00174-1.

[3] X. Zhang, M. Bujarbaruah, and F. Borrelli, "Near-optimal rapid MPC using neural networks: A primal-dual policy learning framework," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 5, pp. 2102–2114, 2021, doi: 10.1109/TCST.2020.3024571.

[4] N. Mansard, A. Del Prete, M. Geisert, S. Tonneau, and O. Stasse, "Using a memory of motion to efficiently warm-start a nonlinear predictive controller," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 2986–2993, doi: 10.1109/ICRA.2018.8463154.

[5] J. Drgoňa, A. Tuor, and D. Vrabie, "Learning constrained parametric differentiable predictive control policies with guarantees," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 54, no. 6, pp. 3596–3607, 2024, doi: 10.1109/TSMC.2024.3368026.

[6] Y. Min and N. Azizan, "HardNet: Hard-constrained neural networks with universal approximation guarantees," in *NeurIPS Workshop on Constrained Machine Learning*, 2025.

[7] L. Zhang and P. N. Suganthan, "A comprehensive evaluation of random vector functional link networks," *Inf. Sci.*, vols. 367–368, pp. 1094–1105, 2016, doi: 10.1016/j.ins.2015.09.025.

[8] R. Verschueren *et al.*, "acados—a modular open-source framework for fast embedded optimal control," *Math. Program. Comput.*, vol. 14, no. 1, pp. 147–183, 2022, doi: 10.1007/s12532-021-00208-8.

[9] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017, doi: 10.1109/TAC.2016.2638961.

[10] K. Choi, Y. Kim, K. S. Kim, and S. K. Kim, "Real-time optimal torque control of interior permanent magnet synchronous motors based on a numerical optimization technique," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 4, pp. 1815–1822, 2021, doi: 10.1109/TCST.2020.3006900.